

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets

QUESTIONS: 05

Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

<i>Criteria</i>	Problem Understanding (2)	Algorithm Design (3)	Use of Control Structures (2)	Pseudocode Structure (1)	Completeness (2)	<i>Total(10)</i>
<i>Weightage</i>	20%	30%	20%	10%	20%	<i>100%</i>
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						
<i>Q4</i>						
<i>Q5</i>						

Code of Conduct:

I G.Hima Bindu(192511377) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.Hima Bindu

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Q1. Dataset: Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question:

Design pseudocode for the **Apriori algorithm** to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration.

Dataset: O1: {Mobile, Earphones}, O2: {Mobile, Charger}, O3: {Earphones, Charger}, O4: {Mobile, Earphones, Charger}, O5: {Mobile, Earphones}

ALGORITHM Apriori(Transactions T, min_support)

INPUT: Transaction database T, minimum support threshold min_support

OUTPUT: All frequent itemsets L

BEGIN

 // Step 1: Generate frequent 1-itemsets

 C1 = {all distinct items in T} // candidate 1-itemsets

 FOR each item i in C1:

 support(i) = COUNT(transactions containing i) / |T|

 END FOR

 L1 = { i in C1 | support(i) >= min_support } // prune infrequent items

 L = L1

 k = 2

// Step 2: Iteratively generate and test candidates

WHILE L_{k-1} is not empty DO

 // --- Candidate generation (self-join) ---

$C_k = \text{JOIN}(L_{k-1}, L_{k-1})$ // join frequent $(k-1)$ -itemsets that

 // share their first $k-2$ items

 // --- Candidate pruning (Apriori property) ---

 FOR each candidate c in C_k :

 FOR each $(k-1)$ -subset s of c :

 IF s is NOT in L_{k-1} THEN

 REMOVE c from C_k // any subset of a frequent itemset

 END IF // must itself be frequent

 END FOR

 END FOR

 // --- Support counting ---

 FOR each transaction t in T :

 FOR each candidate c in C_k :

 IF c is subset of t THEN

$\text{count}[c] = \text{count}[c] + 1$

 END IF

 END FOR

 END FOR

$L_k = \{ c \text{ in } C_k \mid \text{count}[c] / |T| \geq \text{min_support} \}$

L = L UNION L_k

k = k + 1

END WHILE

RETURN L

END

Illustration on the dataset (min_support = 40%, i.e. count ≥ 2 of 5 orders):

- C₁ = {Mobile, Earphones, Charger}; counts: Mobile=4, Earphones=4, Charger=3 → **L₁ = all three** (all ≥ 40%)
- C₂ = join(L₁, L₁) = {Mobile-Earphones, Mobile-Charger, Earphones-Charger}; no pruning needed since all 1-item subsets are in L₁ Counts: Mobile-Earphones = 3 (O₁, O₄, O₅), Mobile-Charger = 2 (O₂, O₄), Earphones-Charger = 2 (O₃, O₄) → **L₂ = all three** (all ≥ 40%)
- C₃ = join(L₂, L₂) = {Mobile-Earphones-Charger}; pruning check: all three 2-item subsets (Mobile-Earphones, Mobile-Charger, Earphones-Charger) are in L₂, so the candidate survives pruning Count = 1 (O₄ only) = 20% < 40% → **L₃ = empty**, loop terminates.

Final frequent itemsets: L₁ ∪ L₂ as listed above.

Q2. Dataset: Library Book Borrowing Records

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question:

Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. (BL4)

Dataset: B1: {Data Science, Python}, B2: {Python, Machine Learning}, B3: {Data Science, Machine Learning}, B4: {Data Science, Python, Machine Learning}, B5: {Python, Machine Learning}

ALGORITHM FPGrowth(Transactions T, min_support)

INPUT: Transaction database T, minimum support threshold

OUTPUT: All frequent itemsets

BEGIN

```

// --- Phase 1: Build the FP-Tree ---
FOR each item i in T:
    count[i] = COUNT(transactions containing i)
END FOR
F = { i | count[i] >= min_support }           // frequent items
SORT F in descending order of count           // ordering used for tree construction

CREATE root node R (empty, null label)

FOR each transaction t in T:
    filtered_t = ITEMS in t that are also in F, SORTED by F's order
    CURRENT = R
    FOR each item i in filtered_t:
        IF CURRENT has a child node named i THEN
            INCREMENT count of that child
        ELSE
            CREATE new child node named i with count = 1
            LINK it into the header table's node-link chain for item i
        END IF
        CURRENT = child node named i
    END FOR
END FOR

// --- Phase 2: Mine the FP-Tree recursively (pattern growth, no candidate generation) ---
FUNCTION MineTree(Tree, alpha):                // alpha = current suffix itemset
    IF Tree contains a single path P THEN
        FOR each combination (comb) of nodes in P:
            GENERATE pattern (comb UNION alpha) with support = MIN(count of nodes in
comb)
        END FOR
    ELSE
        FOR each item ai in the header table of Tree (bottom-up) DO
            pattern beta = ai UNION alpha
            support(beta) = SUM of counts of ai's nodes via node-links
            OUTPUT beta as frequent
            CONSTRUCT conditional pattern base for ai (prefix paths ending at ai)
            CONSTRUCT conditional FP-Tree from that pattern base
            IF conditional FP-Tree is not empty THEN
                MineTree(conditional FP-Tree, beta) // recursive call
            END IF
        END FOR
    END IF
END FUNCTION

RETURN MineTree(FP-Tree, {})
END

```

Applying to the dataset (min_support = 40%, count ≥ 2):
Item counts: Python = 4, Machine Learning = 3, Data Science = 3 → order for the tree:
Python > Machine Learning > Data Science.

Reordered transactions: B1: {Python, Data Science}, B2: {Python, Machine Learning}, B3: {Machine Learning, Data Science}, B4: {Python, Machine Learning, Data Science}, B5: {Python, Machine Learning}

Building the tree: root → Python(4) branches into Machine Learning(3) and Data Science(1); the Machine Learning(3) branch further splits into Data Science(1) and an end (2); B3's {Machine Learning, Data Science} forms its own root-level branch, giving Machine Learning(1)→Data Science(1) separately. No candidate itemsets are ever generated — the tree's structure itself encodes the frequent co-occurrences, and mining works by extracting conditional pattern bases (e.g., the prefix paths ending at Data Science) rather than testing candidate combinations, which is why FP-Growth avoids the expensive candidate generate-and-test step central to Apriori.

Q3. Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Question:

Write pseudocode to generate **association rules** from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. (BL3)

Given frequent itemsets and support counts: {Internet}=4, {Streaming}=4, {Cloud Storage}=3, {Internet, Streaming}=3, {Streaming, Cloud Storage}=2

ALGORITHM GenerateRules(FrequentItemsets L, min_confidence)

INPUT: Set of frequent itemsets L with their support counts, min_confidence threshold

OUTPUT: All strong association rules

BEGIN

Rules = empty set

FOR each frequent itemset f in L WHERE |f| >= 2 DO

 SubsetList = ALL non-empty proper subsets of f

 FOR each subset s in SubsetList DO

 remainder = f - s // items in f but not in s

 IF remainder is not empty THEN

 confidence = support_count(f) / support_count(s) // Step: confidence calc

 // Step: filtering

 IF confidence >= min_confidence THEN

 ADD rule (s -> remainder) with its support and confidence to Rules

 END IF

 END IF

 END FOR

END FOR

RETURN Rules

END

Applying to the dataset (assume min_confidence = 70%, total transactions = 4, since {Internet}=4 is the maximum count shown):

- Itemset {Internet, Streaming} = 3:
 - Rule Internet → Streaming: confidence = $3/4 = 75\%$ ✓ ($\geq 70\%$, kept)
 - Rule Streaming → Internet: confidence = $3/4 = 75\%$ ✓ ($\geq 70\%$, kept)
- Itemset {Streaming, Cloud Storage} = 2:
 - Rule Streaming → Cloud Storage: confidence = $2/4 = 50\%$ ✗ (filtered out)
 - Rule Cloud Storage → Streaming: confidence = $2/3 = 66.7\%$ ✗ (filtered out)

Resulting strong rules: Internet → Streaming (75%), Streaming → Internet (75%).

Q4. Dataset: Student Course Enrollment

Student ID Courses Enrolled

S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question:

Design pseudocode for **constraint-based frequent itemset mining** where only itemsets containing the course **"Python"** are considered during the mining process. Explain how the constraint affects candidate generation and pruning. (BL4)

Dataset: S1: {Python, Database}, S2: {Python, AI}, S3: {Database, AI}, S4: {Python, Database, AI}, S5: {Python}

ALGORITHM ConstrainedApriori(Transactions T, min_support, required_item)

INPUT: Transaction database T, min_support, an item that **MUST** appear in every valid itemset

OUTPUT: Frequent itemsets that satisfy the constraint

BEGIN

 // Step 1: generate 1-itemsets as usual (constraint doesn't apply to singletons yet)

 C1 = {all distinct items in T}

 FOR each item i in C1:

 support(i) = COUNT(transactions containing i) / |T|

 END FOR

 L1 = { i in C1 | support(i) $\geq$ min_support }

 IF required_item NOT in L1 THEN

 RETURN empty set // constraint can never be satisfied

 END IF


```

L = { required_item }           // only keep the required singleton
k = 2

WHILE Lk-1 is not empty DO
    Ck = JOIN(Lk-1, Lk-1)

    // --- Constraint-based pruning (applied BEFORE support counting) ---
    FOR each candidate c in Ck:
        IF required_item NOT in c THEN
            REMOVE c from Ck           // discard early: skip counting entirely
        END IF
        FOR each (k-1)-subset s of c:
            IF s not in Lk-1 THEN
                REMOVE c from Ck       // standard Apriori anti-monotone pruning
            END IF
        END FOR
    END FOR

    // Support counting only on the surviving, constraint-satisfying candidates
    FOR each transaction t in T:
        FOR each candidate c in Ck:
            IF c is subset of t THEN
                count[c] = count[c] + 1
            END IF
        END FOR
    END FOR

    Lk = { c in Ck | count[c] / |T| >= min_support }
    L = L UNION Lk
    k = k + 1
END WHILE

RETURN L
END

```

Applying to the dataset (min_support = 40%, count ≥ 2 of 5 students):

- L₁ (constraint check): Python = 4 (80%) ✓ — required item is frequent, so proceed.
- C₂ candidates restricted to those containing Python: {Python, Database}, {Python, AI} Counts: Python-Database = 2 (S₁, S₄) = 40% ✓; Python-AI = 2 (S₂, S₄) = 40% ✓ → both frequent (Note: {Database, AI} is never even generated as a candidate because it doesn't contain Python — this is the constraint pruning at work.)
- C₃ candidate: {Python, Database, AI}, count = 1 (S₄) = 20% ✗ → not frequent, loop ends.

How the constraint affects candidate generation and pruning: Normally Apriori would generate {Database, AI} as a valid 2-itemset candidate and spend a full pass counting its support. Because the constraint requires every itemset to contain "Python," that candidate is discarded **before** the support-counting scan even runs — this is a form of constraint push-down, which is cheaper than generating the itemset and filtering results afterward. It reduces both the candidate set size at every level and the number of transaction scans needed.

Q5. Dataset: Hospital Treatment Hierarchy

Patient ID	Treatments Received
P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question:

Develop pseudocode for **multilevel association rule mining** using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

Hierarchy: Healthcare (top level) $\rightarrow$ {Cardiology, Orthopedics} (mid level) $\rightarrow$ {ECG} (specific, child of Cardiology)

Dataset: P1: {Healthcare, Cardiology, ECG}, P2: {Healthcare, Orthopedics}, P3: {Healthcare, Cardiology}, P4: {Healthcare, Cardiology, ECG}, P5: {Healthcare, Orthopedics}

ALGORITHM MultilevelAssociationMining(Transactions T, Hierarchy H, min_support[], min_confidence)

INPUT: Transaction database T, concept hierarchy H (top $\rightarrow$ mid $\rightarrow$ specific),

a minimum support threshold per level, min_confidence

OUTPUT: Association rules at each hierarchy level

BEGIN

levels = H.levels_top_to_bottom() // e.g. [Level1, Level2, Level3]

FOR each level Lv in levels DO

// Step 1: generalize transactions to this level

T_level = empty list

FOR each transaction t in T:

t_generalized = MAP each item in t to its ancestor at level Lv

REMOVE duplicate items within t_generalized

```

    APPEND t_generalized to T_level
END FOR

// Step 2: run standard frequent itemset mining at this level
FrequentItemsets[Lv] = Apriori(T_level, min_support[Lv])

// Step 3: generate rules from this level's frequent itemsets
FOR each itemset f in FrequentItemsets[Lv] WHERE |f| >= 2 DO
    FOR each non-empty proper subset s of f DO
        confidence = support_count(f) / support_count(s)
        IF confidence >= min_confidence THEN
            OUTPUT rule (s -> f - s) at level Lv
        END IF
    END FOR
END FOR
END FOR

```

```

// Step 4 (optional refinement): only descend into a branch at the next level
// if its parent node was frequent at this level (level-cross filtering)
PRUNE items at level Lv+1 whose parent is NOT in FrequentItemsets[Lv]
END FOR

```

```

RETURN all rules generated across levels
END

```

Applying to the dataset (min_support = 40%, count ≥ 2 of 5 patients):

- **Top level (Healthcare only):** all 5 patients have Healthcare → trivially 100% support, no useful rule (single item, too general).

- **Mid level (Cardiology, Orthopedics):** Cardiology = 3 (P1,P3,P4) = 60% ✓; Orthopedics = 2 (P2,P5) = 40% ✓. Since Healthcare co-occurs with everything, rule Healthcare → Cardiology has support 60%, confidence 60% (weak/generic and usually filtered as redundant since Healthcare is an ancestor of Cardiology).
- **Specific level (ECG, filtered by level-cross pruning to only descend under Cardiology since it was frequent at the mid-level):** ECG = 2 (P1, P4), both also containing Cardiology → rule Cardiology → ECG: support = 2/5 = 40%, confidence = 2/3 = 66.7%.

How rules are generated across levels & why hierarchies improve discovery: Instead of mining one flat set of items, the hierarchy lets the algorithm generalize specific treatments (ECG) up to broader categories (Cardiology, Healthcare) and mine at each layer separately. This surfaces patterns that would be missed at a single level — e.g., "Cardiology → ECG" is a meaningful mid-to-specific pattern that wouldn't appear if ECG were mined flat against the whole population (its raw support of 40% might look identical to unrelated coincidental pairings without the hierarchy providing context). The level-cross pruning step (Step 4) also cuts wasted work: since Orthopedics patients never receive ECG, the algorithm doesn't need to generate or test Orthopedics-ECG candidates at all.